Guía completa — Ready-to-use implementations

YeisenK

ICPC

Grafos

`2sat.cpp`

2-SAT (Satisfacibilidad booleana 2-literal)

Resuelve el problema de satisfacibilidad booleana cuando cada cláusula tiene exactamente 2 literales. Construye un grafo de implicaciones y encuentra las componentes fuertemente conexas (SCC) con Tarjan o Kosaraju. Si una variable y su negación caen en la misma SCC, no hay solución.

Complejidad: `O(V + E)`

Cuándo usarlo:
Restricciones del tipo 'A o B debe ser verdadero/falso'. Coloración de grafos, asignación de orientaciones a aristas, scheduling con conflictos mutuamente excluyentes.

Notas de implementación:
Siempre hacer shuffle de las variables antes de correr SCC para evitar casos degenerados. La solución se reconstruye leyendo el orden topológico inverso de las SCCs.

`BellmanFord.cc`

Bellman-Ford

Calcula la distancia mínima desde una fuente a todos los vértices, permitiendo aristas de peso negativo. Detecta ciclos de peso negativo alcanzables desde la fuente ejecutando una iteración extra: si aún hay relajaciones, existe un ciclo negativo.

Complejidad: `O(V · E)`

Cuándo usarlo:
Grafos con aristas de peso negativo donde Dijkstra falla. Detección de ciclos negativos (arbitrage en finanzas, restricciones en scheduling). Grafos dispersos con pesos mixtos.

Notas de implementación:
Para grafos densos usar Floyd-Warshall. Si solo importa detectar ciclos negativos (no distancias), basta con V iteraciones.

`DijkstraFloyd.cc / FastDijkstra.cc`

Dijkstra + Floyd-Warshall

Dijkstra encuentra caminos mínimos desde una fuente en grafos con pesos no negativos usando una cola de prioridad. FloydWarshall calcula todos los pares de caminos mínimos en $O(V^3)$ y detecta ciclos negativos si d[i][i] < 0.

Complejidad: `Dijkstra: O((V+E) log V)  |  Floyd: O(V³)`

Cuándo usarlo:
Dijkstra: camino mínimo desde un nodo, grafos grandes con pesos >= 0. Floyd-Warshall: grafos pequeños (V <= 400), todos los pares, detección de ciclos negativos.

Notas de implementación:
FastDijkstra usa SPFA (Bellman-Ford con cola) para grafos donde las aristas pueden ser negativas pero no hay ciclos negativos.

`Kruskal.cc / Prim.cc`

MST — Kruskal y Prim

Kruskal ordena las aristas por peso y usa Union-Find para agregar la de menor peso que no forme ciclo. Prim crece el árbol desde un nodo, seleccionando en cada paso la arista más barata que conecta un nodo nuevo.

Complejidad: `Kruskal: O(E log E)  |  Prim: O(E log V)`

Cuándo usarlo:
Kruskal: grafos dispersos, $E << V^2$. Prim con heap: grafos densos. Ambos producen el árbol de expansión mínima (MST).

Notas de implementación:
Kruskal es más fácil de implementar en competencias. Si hay múltiples componentes, el resultado es un bosque.

`EulerianPath.cc / eulerian.cpp / Euler.cpp`

Camino / Circuito Euleriano

Encuentra un camino que recorra cada arista exactamente una vez (camino euleriano) o un circuito que regrese al origen (circuito euleriano). Usa el algoritmo de Hierholzer. Requiere verificar condiciones de grado: para circuito en grafo no dirigido, todos los vértices tienen grado par.

Complejidad: `O(V + E)`

Cuándo usarlo:
Problemas de rutas que deben cubrir todas las conexiones exactamente una vez: rutas postales, dibujo sin levantar el lápiz, reconstrucción de secuencias de ADN (De Bruijn).

Notas de implementación:
En grafo dirigido: circuito requiere in-degree == out-degree para todo vértice. Camino requiere exactamente un vértice con out-degree = in-degree + 1 (inicio) y uno con in-degree = out-degree + 1 (fin).

`TopologicalSort.cc`

Ordenamiento topológico

Ordena los vértices de un DAG (grafo dirigido acíclico) tal que para toda arista (u, v), u aparece antes que v. Implementado con DFS (al finalizar visitar un nodo, empujarlo a la pila) o con BFS usando grados de entrada (algoritmo de Kahn).

Complejidad: `O(V + E)`

Cuándo usarlo:
Dependencias entre tareas, compilación incremental, detección de ciclos en grafos dirigidos, DP sobre DAGs.

Notas de implementación:
Si el grafo tiene ciclos, no existe orden topológico. Kahn detecta esto: si la cola se vacía antes de procesar todos los nodos, hay un ciclo.

`LCA.cc`

LCA — Ancestro Común Más Bajo

Encuentra el ancestro común más profundo de dos nodos en un árbol. Usa binary lifting: dp[v][k] = ancestro 2^k pasos arriba de v. Permite responder queries de distancia y caminos en árboles.

Complejidad: `O(n log n) preproceso | O(log n) query`

Cuándo usarlo:
Distancia entre nodos en árbol, queries de caminos, problemas de árbol con actualización de ancestros, optimización de DP sobre árboles.

Notas de implementación:
Variante con Euler tour + RMQ (Sparse Table) da O(1) query pero requiere más memoria. Usar LCA binario para implementación rápida en competencias.

`SCC.cc / scc.cpp / graph_scc.cpp`

SCC — Componentes Fuertemente Conexas

Encuentra todas las componentes fuertemente conexas de un grafo dirigido usando Kosaraju (dos DFS: uno en el grafo original, otro en el transpuesto) o Tarjan (un DFS con pila y low-link values).

Complejidad: `O(V + E)`

Cuándo usarlo:
2-SAT, condensación de grafos, detección de ciclos en grafos dirigidos, encontrar bloques de código con dependencias circulares.

Notas de implementación:
Tarjan es más rápido en la práctica (un solo DFS). Kosaraju es más fácil de recordar e implementar correctamente bajo presión.

`bridges.cpp / graph_bcc.cpp`

Puentes y Componentes Biconexas

Un puente es una arista cuya eliminación desconecta el grafo. Una componente biconexa es un

subgrafo maximal sin puentes. Se calculan con un DFS modificado que lleva valores disc[] y low[].

Complejidad: `O(V + E)`

Cuándo usarlo:
Encontrar puntos críticos de fallo en redes, problemas de conectividad, reducir grafos para simplificar estructura.

Notas de implementación:
Los puntos de articulación (vértices cuya eliminación desconecta) se calculan con el mismo DFS. Cuidado con aristas múltiples (multiedges) al detectar puentes.

`ds_centroid_decomposition.cpp`

Descomposición por centroide

Divide recursivamente un árbol por su centroide (nodo cuya eliminación deja componentes de tamaño <= n/2). Permite resolver queries de caminos en árboles en O(log n) por query, ya que todo camino pasa por exactamente un centroide.

Complejidad: `O(n log n) total para queries de caminos`

Cuándo usarlo:
Contar pares de nodos con distancia exactamente k, problemas de caminos en árboles que son difíciles de resolver directamente con DP.

Notas de implementación:
La profundidad de la recursión es O(log n). Requiere marcar centroides como 'eliminados' para no visitarlos de nuevo.

`ds_heavy_light_decomposition.cpp`

Heavy-Light Decomposition (HLD)

Descompone un árbol en cadenas 'pesadas' (siguiendo el hijo con el subárbol más grande) y cadenas 'ligeras'. Permite aplicar estructuras de datos de rango (segment tree, BIT) sobre caminos en el árbol.

Complejidad: `O(log² n) por query con segment tree`

Cuándo usarlo:
Queries de suma/mínimo/máximo sobre caminos en árboles con actualizaciones, problemas que requieren range queries sobre árboles.

Notas de implementación:
Combinado con segment tree lazy da O(log² n). Puede usarse también sobre el arreglo de Euler tour para queries de subárbol.

Flujo de redes

`MaxFlow.cc / graph_maxflow.cpp`

Flujo máximo — Ford-Fulkerson / Edmonds-Karp

Calcula el flujo máximo en una red desde una fuente s a un sumidero t. Edmonds-Karp usa BFS

para encontrar el camino de aumento más corto (en número de aristas), garantizando $O(VE^2)$. El teorema max-flow min-cut garantiza que el flujo máximo iguala el corte mínimo.

Complejidad: `O(V · E²) Edmonds-Karp`

Cuándo usarlo:
Matching bipartito (se reduce a flujo), problemas de asignación, circulación de flujo, problemas de corte en redes.

Notas de implementación:
Para grafos grandes usar Dinic. Ford-Fulkerson con DFS puede ser exponencial si los pesos son irracionales.

`Dinic.cc / dinic.cpp / graph_dinic.cpp`

Flujo máximo — Algoritmo de Dinic

Construye un grafo en capas (BFS desde s) y envía flujo bloqueante por DFS. La combinación da una complejidad mucho mejor que Edmonds-Karp en la práctica. Es la implementación estándar de flujo máximo en competencias.

Complejidad: `O(V² · E)  |  O(E√V) en grafos unitarios`

Cuándo usarlo:
Flujo máximo en general. En matching bipartito (grafo unitario) corre en $O(E\sqrt{V})$, superando a Hopcroft-Karp en implementación.

Notas de implementación:
Siempre usar Dinic en lugar de Ford-Fulkerson/Edmonds-Karp. Asegurarse de que la función de BFS use el grafo de capacidades residuales.

`MinCostMaxFlow.cc / graph_mincost_maxflow.cpp / mcmf4.cpp`

Min-Cost Max-Flow (MCMF)

Extiende el flujo máximo con costos en las aristas. Encuentra el flujo máximo de costo mínimo usando SPFA (Bellman-Ford con cola) o Dijkstra con potenciales de Johnson para reducir a costos no negativos.

Complejidad: `O(V · E · maxFlow) con SPFA`

Cuándo usarlo:
Asignación óptima de recursos con costos, matching con pesos mínimos, problemas de transporte y distribución donde importa el costo total.

Notas de implementación:
mcmf4.cpp usa SPFA que puede ser lento con ciclos de costo negativo. Preferir versión con potenciales de Johnson (Dijkstra) para mejor rendimiento.

`PushRelabel.cc`

Flujo máximo — Push-Relabel

Algoritmo alternativo para flujo máximo basado en preflows y alturas. En lugar de buscar caminos de aumento, empuja flujo localmente desde nodos con exceso. Asintóticamente más rápido que Dinic

en grafos muy densos.

Complejidad: `O(V² · √E)`

Cuándo usarlo:
Grafos extremadamente densos donde Dinic es lento, problemas de flujo máximo de gran escala.

Notas de implementación:
Más difícil de implementar correctamente. En la mayoría de problemas de competencia, Dinic es suficiente y más simple.

`MinCut.cc / mincut.cpp`

Corte mínimo (Min-Cut)

Por el teorema max-flow min-cut, el valor del flujo máximo iguala la capacidad del corte mínimo. Para encontrar el corte: correr flujo máximo, luego BFS en el grafo residual desde s; los vértices alcanzables forman el lado S del corte.

Complejidad: `O(V · E²) o el costo del flujo máximo`

Cuándo usarlo:
Separación mínima de redes, problemas de conectividad, image segmentation, problemas de proyecto-selección.

Notas de implementación:
Stoer-Wagner (stoer-wagner.cpp) calcula el min-cut global (sin fuente/sumidero fijos) en $O(V^3)$.

`MaxBipartiteMatching.cc / bipartite.cpp / graph_hopcroft_karp.cpp`

Matching bipartito máximo (Hopcroft-Karp)

Encuentra el emparejamiento máximo en un grafo bipartito. Hopcroft-Karp usa BFS para encontrar múltiples caminos de aumento disjuntos simultáneamente, mejorando el clásico O(VE) de augmenting paths.

Complejidad: `O(E · √V)`

Cuándo usarlo:
Asignación de tareas a trabajadores, scheduling, cobertura mínima de vértices (por König: |min vertex cover| = |max matching| en bipartito).

Notas de implementación:
Para matching con pesos, usar MinCostMatching. Para grafos generales (no bipartitos), usar general-matching.cpp (Blossom algorithm).

`general-matching.cpp`

Matching general (Algoritmo de Blossom)

Calcula el matching máximo en grafos generales (no necesariamente bipartitos). Implementa el algoritmo de Edmonds con contracción de flores (blossoms): ciclos impares que se contraen en un solo nodo.

Complejidad: `O(V³)`

Cuándo usarlo:

Matching en grafos no bipartitos, problemas donde los vértices forman ciclos de longitud impar.

Notas de implementación:
Mucho más complejo de implementar que el bipartito. Solo usar si el grafo no es bipartito.

`MinCostMatching.cc / bipartite-mincost.cpp / KM.cpp`

Matching bipartito de mínimo costo (Kuhn-Munkres / Hungarian)

Encuentra el matching perfecto de costo mínimo (o máximo) en un grafo bipartito ponderado. KM.cpp implementa el algoritmo húngaro. stablemp.cpp resuelve el matching estable (Gale-Shapley).

Complejidad: $O(V^3)$

Cuándo usarlo:
Asignación óptima de recursos con costos/preferencias, problemas de asignación donde cada trabajador debe ser asignado a exactamente una tarea.

Notas de implementación:
Requiere grafo bipartito con matching perfecto posible. Para matching máximo de peso (no perfecto), usar MCMF.

`GraphCutInference.cc`

Graph Cut Inference

Resuelve problemas de inferencia MAP en campos aleatorios de Markov con potenciales de pares submodulares usando corte mínimo en grafos. Transforma la energía de optimización en una red de flujo.

Complejidad: $O(V^2 \cdot E)$

Cuándo usarlo:
Visión computacional (segmentación de imágenes), problemas de optimización combinatoria que se pueden expresar como QPBO (Pseudo-Boolean Optimization).

Notas de implementación:
Solo aplica cuando los potenciales de pares son submodulares. De lo contrario el problema es NP-hard.

Estructuras

`BIT.cc / BIT.cpp / BIT-range.cpp`

BIT — Binary Indexed Tree (Fenwick Tree)

Estructura de datos compacta para sumas de prefijo con actualización de punto. BIT-range.cpp extiende a actualizaciones de rango. Cada posición almacena la suma de un rango de longitud potencia de 2, determinado por el bit menos significativo.

Complejidad: `O(log n) update/query | O(n) construcción`

Cuándo usarlo:
Sumas de prefijo dinámicas, inversiones en un arreglo, frecuencias acumuladas, problemas de

conteo con compresión de coordenadas.

Notas de implementación:
Más rápido y simple que un segment tree para sumas puras. Para queries más generales (mínimo, máximo), usar segment tree.

`segtree.cpp / SegmentTreeLazy.java`

Segment Tree (con lazy propagation)

Árbol binario para queries de rango (suma, mínimo, máximo, GCD, etc.) y actualizaciones de punto o rango. La versión lazy difiere las actualizaciones de rango hasta que son necesarias, manteniendo O(log n) por operación.

Complejidad: `O(log n) query/update | O(n) construcción`

Cuándo usarlo:
Range minimum/maximum/sum queries con actualizaciones, problemas de intervalos con asignación en rango, inversiones con actualizaciones.

Notas de implementación:
SegmentTreeLazy.java implementa la versión con lazy en Java. Para problemas sin actualizaciones de rango, la versión simple sin lazy es suficiente.

`UnionFind.cc`

Union-Find (Disjoint Set Union)

Estructura de datos para conjuntos disjuntos con las operaciones find (encontrar representante) y union (fusionar conjuntos). Usa compresión de rutas y unión por rango para lograr el inverso de Ackermann α(n).

Complejidad: `O(α(n)) por operación (casi O(1))`

Cuándo usarlo:
Conectividad dinámica, Kruskal MST, detección de ciclos en grafos no dirigidos, problemas de equivalencia y agrupación.

Notas de implementación:
α(n) < 5 para cualquier n práctico — esencialmente O(1). Uno de los bloques de construcción más usados en competencias.

`KDTree.cc / KDTree.cpp`

KD-Tree

Estructura de datos espacial para puntos en k dimensiones. Divide el espacio recursivamente por la mediana del eje con mayor varianza. Soporta consultas de vecino más cercano (KNN) y rango rectangular.

Complejidad: `O(√n) query esperado | O(n log n) construcción`

Cuándo usarlo:
Vecino más cercano en 2D/3D, problemas de proximidad geométrica, consultas de rango en múltiples dimensiones.

Notas de implementación:
En el peor caso degrada a O(n). Funciona bien para k pequeño (2-3 dimensiones). Para datos dinámicos considerar R-trees.

`SBT.cpp`

Size Balanced Tree (SBT)

Árbol de búsqueda binaria auto-balanceado que mantiene el balance usando el tamaño de los subárboles. Soporta operaciones de orden (k-ésimo elemento, rank) además de inserción, eliminación y búsqueda.

Complejidad: `O(log n) todas las operaciones`

Cuándo usarlo:
Cuando se necesita un BST con operaciones de orden estadístico y no se dispone de policy-based data structures de GCC.

Notas de implementación:
ds_pbds.cpp expone las policy-based data structures de GCC (order statistics tree) que son más simples en competencias modernas.

`ds_pbds.cpp`

PBDS — Policy-Based Data Structures (GCC)

Expone las estructuras de datos de GCC: order statistics tree (find_by_order, order_of_key), rope, y trie. Son extensiones no estándar disponibles en GCC que proveen sets/maps con estadísticas de orden.

Complejidad: `O(log n) todas las operaciones`

Cuándo usarlo:
k-ésimo elemento, rank de un valor, operaciones de conjunto con índice. Solo disponible en compiladores GCC.

Notas de implementación:
__gnu_pbds::tree con tree_order_statistics_node_update es el más útil: permite rank y select en O(log n).

`ds_treap.cpp`

Treap (árbol + heap aleatorizado)

BST que asigna prioridades aleatorias a cada nodo para mantener la propiedad de heap (balance esperado). Soporta split y merge en O(log n), permitiendo implementar secuencias dinámicas con operaciones de rango.

Complejidad: `O(log n) esperado todas las operaciones`

Cuándo usarlo:
BST dinámico con split/merge, implementar arrays dinámicos con operaciones de rango en O(log n), persistencia.

Notas de implementación:
Más simple que un red-black tree. ds_splay_tree.cpp y ds_link_cut_tree.cpp ofrecen alternativas con

propiedades diferentes.

`ds_link_cut_tree.cpp`

Link-Cut Tree

Estructura de datos para representar un bosque de árboles con soporte para link (unir dos árboles), cut (separar en dos), y path queries (agregados sobre el camino entre dos nodos). Internamente usa splay trees con preferred paths.

Complejidad: `O(log n) amortizado por operación`

Cuándo usarlo:
Conectividad dinámica en árboles, queries de caminos con actualizaciones dinámicas de la estructura del árbol.

Notas de implementación:
Una de las estructuras más complejas del codebook. Solo necesaria para problemas con modificaciones dinámicas de la topología del árbol.

`ds_dynamic_convex_hull.cpp`

Convex Hull dinámico

Mantiene el convex hull de un conjunto de puntos 2D con inserciones dinámicas, soportando consultas del máximo de funciones lineales (y = mx + b) evaluadas en un punto. Implementado con un set ordenado.

Complejidad: `O(log n) inserción y query`

Cuándo usarlo:
Convex Hull Trick dinámico cuando las pendientes o los puntos de consulta no son monótonos, optimización de DP con pendientes arbitrarias.

Notas de implementación:
Para el caso estático o con pendientes/queries monótonas, chtrick.cpp es más simple y eficiente.

`ansv.cpp`

ANSV — All Nearest Smaller Values

Para cada elemento de un arreglo, encuentra el elemento más cercano a la izquierda y derecha que sea estrictamente menor. Se resuelve con una pila monotónica en un solo recorrido.

Complejidad: `O(n)`

Cuándo usarlo:
Área máxima bajo histograma, rangos de dominancia, cálculo de árboles cartesianos, problemas de pila monotónica.

Notas de implementación:
Bloque fundamental para el cálculo del árbol cartesiano y varios problemas de geometría con arreglos.

`monotone.cpp`

Pila / Cola monotónica

Estructura que mantiene una secuencia de elementos en orden monótono. La pila monotónica resuelve next greater element en O(n). La cola monotónica resuelve mínimo/máximo en ventana deslizante en O(n).

Complejidad: `O(n) total`

Cuándo usarlo:
Máximo/mínimo en ventana deslizante, next greater/smaller element, optimizaciones de DP con transiciones de ventana.

Notas de implementación:
Uno de los trucos de eficiencia más subestimados. Convierte soluciones $O(n^2)$ obvias en O(n).

Strings

`KMP.cc / KMP.cpp / string_KMP.cpp`

KMP — Knuth-Morris-Pratt

Algoritmo de búsqueda de patrón en texto en tiempo lineal. Preprocesa el patrón para construir la función de fallo (failure function / π-array): la longitud del prefijo más largo que también es sufijo de cada prefijo del patrón.

Complejidad: `O(n + m) búsqueda | O(m) preproceso`

Cuándo usarlo:
Búsqueda de patrón en texto, contar ocurrencias, encontrar el período de una cadena, autocorrelación de strings.

Notas de implementación:
La función π (failure function) tiene múltiples usos más allá de la búsqueda. string_z_function.cpp calcula la Z-function, equivalente pero con implementación diferente.

`string_z_function.cpp`

Z-function

Para cada posición i, calcula la longitud del substring más largo que empieza en i y coincide con un prefijo de la cadena. Es equivalente a KMP pero con implementación y usos ligeramente diferentes.

Complejidad: `O(n)`

Cuándo usarlo:
Búsqueda de patrón (concatenar patrón + '$' + texto), períodos de cadenas, problemas de comparación de prefijos.

Notas de implementación:
Z[0] = n por convención. Para búsqueda de patrón P en texto T: calcular Z de P+'$'+T; posiciones donde Z[i] >= |P| son ocurrencias.

`SuffixArray.cc / suffix-array.cpp / string_SA.cpp`

Suffix Array + LCP

Ordena todos los sufijos de una cadena lexicográficamente. Combinado con el arreglo LCP (Longest Common Prefix, construido con el algoritmo de Kasai en O(n)), permite responder queries de subcadenas eficientemente.

Complejidad: `O(n log n) o O(n) construcción | O(n) LCP`

Cuándo usarlo:
Número de subcadenas distintas, substring común más largo (LCS) entre múltiples cadenas, búsqueda de patrón, compresión.

Notas de implementación:
string_SA_nlogn_pear.cpp implementa la variante O(n log n). string_kasai.cpp construye el LCP. Para construcción O(n) usar DC3/Skew.

`string_suffix_automaton.cpp`

Suffix Automaton (SAM)

Autómata de estados finito más pequeño que acepta todos los sufijos de una cadena. Cada estado representa un conjunto de subcadenas. Soporta búsqueda de subcadenas, conteo de ocurrencias y LCS en tiempo lineal.

Complejidad: `O(n) construcción`

Cuándo usarlo:
Problemas avanzados de strings donde el suffix array no es suficiente, conteo de subcadenas distintas en O(n), LCS de múltiples cadenas.

Notas de implementación:
Más potente que el suffix array para algunos problemas pero más complejo de implementar. El número de estados es O(n) y el de transiciones O(n).

`string_ukkonen.cpp`

Suffix Tree (Ukkonen)

Construye el árbol de sufijos de una cadena en tiempo y espacio O(n) usando el algoritmo de Ukkonen. Es la representación comprimida del suffix automaton y permite navegar la estructura de sufijos como un árbol.

Complejidad: `O(n) construcción`

Cuándo usarlo:
Problemas que requieren navegar la estructura árbol de los sufijos, LCA en suffix tree para LCS, problemas que son más naturales en árbol que en arreglo.

Notas de implementación:
Muy difícil de implementar correctamente. En la mayoría de casos, el suffix array + LCP + RMQ es suficiente y más simple.

`string_AC.cpp`

Aho-Corasick

Extiende KMP a múltiples patrones simultáneos. Construye un trie con todos los patrones y añade failure links (como la función π de KMP). Busca todas las ocurrencias de todos los patrones en el texto en un solo recorrido.

Complejidad: `O(Σ|pi|) construcción | O(n) búsqueda`

Cuándo usarlo:
Búsqueda de múltiples patrones en un texto, problemas de matching de diccionario, DP sobre strings donde los estados son patrones.

Notas de implementación:
Los failure links convierten el trie en un autómata finito. Muy útil en combinación con DP (e.g., contar strings de longitud n que evitan ciertos patrones).

`manacher.cpp`

Algoritmo de Manacher

Encuentra la subcadena palíndromica más larga centrada en cada posición de una cadena en tiempo lineal. Extiende inteligentemente el palíndromo usando los ya calculados para evitar recomparaciones.

Complejidad: `O(n)`

Cuándo usarlo:
Palíndromo más largo, contar palíndromos distintos, problemas de simetría en strings.

Notas de implementación:
Insertar '#' entre caracteres convierte el caso par/impar en uno solo. d1[i] = radio del palíndromo impar centrado en i; d2[i] = radio del par.

`LCS.cc`

LCS — Longest Common Subsequence

Calcula la subsecuencia común más larga de dos cadenas usando programación dinámica. dp[i][j] = longitud de LCS de los primeros i caracteres de A y j de B.

Complejidad: `O(n · m)`

Cuándo usarlo:
Diff de archivos, distancia de edición (relacionado), alineación de secuencias biológicas.

Notas de implementación:
Para LCS en O(n log n) cuando los valores son distintos, usar la reducción a LIS (Longest Increasing Subsequence).

`HashWithAnd.cpp`

Hashing de cadenas

Hashing polinomial de subcadenas con módulo. Permite comparar subcadenas en O(1) después de O(n) preproceso. HashWithAnd.cpp usa AND con máscara para modular en lugar de división, siendo más rápido.

Complejidad: `O(n) preproceso | O(1) query`

Cuándo usarlo:
Comparación rápida de subcadenas, búsqueda de patrón en O(n) esperado (Rabin-Karp), detección de palíndromos, sufijos/prefijos iguales.

Notas de implementación:
Usar doble hash (dos bases y módulos diferentes) para reducir colisiones. Módulos primos grandes como 1e9+7 y 1e9+9.

Geometría

`ConvexHull.cc / geom-2d.cpp / Geometry.cc / geom_library.cpp`

Convex Hull 2D

Calcula la envolvente convexa del conjunto mínimo de puntos que contienen a todos los demás. Implementaciones incluyen Graham Scan y Andrew's Monotone Chain (más estable numéricamente). geom_library.cpp agrupa funciones geométricas básicas (punto en polígono, distancia, intersección de segmentos).

Complejidad: `O(n log n)`

Cuándo usarlo:
Punto en convex hull, diámetro de un conjunto de puntos, par de puntos más lejanos, problemas de cobertura convexa.

Notas de implementación:
Andrew's Monotone Chain es preferible por no requerir atan2. Cuidado con la orientación (CCW vs CW) y el manejo de puntos colineales.

`Delaunay.cc / delaunay.cpp / Delaunay.java`

Triangulación de Delaunay

Triangulación de un conjunto de puntos tal que ningún punto esté dentro del circumcírculo de ningún triángulo. Maximiza el ángulo mínimo de todos los triángulos. El grafo dual es el diagrama de Voronoi.

Complejidad: `O(n log n) esperado`

Cuándo usarlo:
Par de puntos más cercanos (aristas del grafo de Delaunay), Voronoi diagram, interpolación, mallas para elementos finitos.

Notas de implementación:
El grafo de Voronoi es el dual de Delaunay. El par de puntos más cercano siempre está conectado por una arista de Delaunay — permite calcular el vecino más cercano eficientemente.

`welzl.cpp`

Círculo mínimo encerrante (Welzl)

Encuentra el círculo de mínimo radio que contiene todos los puntos. Algoritmo aleatorizado de Welzl: si el punto actual no está en el círculo de los puntos anteriores, debe estar en la frontera del círculo óptimo.

Complejidad: `O(n) esperado`

Cuándo usarlo:
Área mínima que cubre un conjunto de puntos, problemas de cobertura con círculos.

Notas de implementación:
Requiere shuffle previo del arreglo de puntos para garantizar O(n) esperado. Sin shuffle, el peor caso es $O(n^2)$.

`LatLong.cc`

Coordenadas geográficas (Lat/Long)

Funciones para trabajar con coordenadas geográficas: distancia entre puntos en la esfera terrestre (fórmula haversine), conversión entre grados y radianes, rumbo (bearing) entre dos puntos.

Complejidad: `O(1) por cálculo`

Cuándo usarlo:
Problemas geográficos con distancias sobre la superficie terrestre (great-circle distance), rutas de navegación.

Notas de implementación:
La fórmula haversine es numéricamente estable para distancias cortas. Para mayor precisión usar Vincenty.

`geom-3d.cpp / Geom3D.java`

Geometría 3D

Operaciones vectoriales en 3D: producto punto, producto cruz, proyección, distancia punto-plano, distancia punto-recta, intersección de planos. JavaGeometry.java y geom-3d.cpp proveen una librería completa.

Complejidad: `O(1) por operación básica`

Cuándo usarlo:
Problemas de geometría en tres dimensiones, cálculo de volúmenes, intersecciones de planos, proyecciones.

Notas de implementación:
El producto cruz $A \times B$ da el vector normal al plano que contiene A y B, con magnitud $|A||B|\sin(\theta)$. Fundamental para geometría 3D.

Matemáticas

`FFT.cc / FFT_new.cc / fft.cpp / math_fft.cpp`

FFT — Fast Fourier Transform

Calcula la transformada discreta de Fourier en O(n log n) usando el algoritmo Cooley-Tukey. Permite multiplicar polinomios (convolución) de grado n en O(n log n) en lugar de $O(n^2)$. FFT_new.cc usa números de punto flotante; math_ntt.cpp usa NTT (Number Theoretic Transform) con módulo primo para resultados exactos.

Complejidad: `O(n log n)`

Cuándo usarlo:
Multiplicación de polinomios grandes, multiplicación de enteros grandes, convolución de secuencias, problemas de conteo con rangos.

Notas de implementación:
NTT (math_ntt.cpp) es más preciso que FFT de punto flotante para enteros. Usar módulos como 998244353 (primo NTT-friendly: $998244353 = 119 \cdot 2^{23} + 1$).

`GaussJordan.cc / gaussian.cpp / ReducedRowEchelonForm.cc / rref.cpp`

Eliminación gaussiana / Gauss-Jordan

Resuelve sistemas de ecuaciones lineales Ax = b. Gaussian.cpp opera sobre reales con pivoteo parcial. rref.cpp / ReducedRowEchelonForm.cc operan sobre GF(2) (campo de dos elementos, aritmética XOR) para álgebra lineal binaria.

Complejidad: $O(n^3)$

Cuándo usarlo:
Sistemas de ecuaciones lineales, inversión de matrices, rango de una matriz. rref sobre GF(2): problemas de interruptores, independencia lineal sobre bits.

Notas de implementación:
rref.cpp es especialmente útil en problemas de juegos y puzzles con operaciones XOR. La versión sobre GF(2) usa bitsets para eficiencia.

`Euclid.cc / Primes.cc / primes.cpp / MillerRabin.cc`

Aritmética de números y primos

Euclid.cc: GCD extendido, inverso modular, ecuaciones diofánticas lineales. Primes.cc/primes.cpp: criba de Eratóstenes. MillerRabin.cc: prueba de primalidad probabilística. totient.cpp: función de Euler φ. math_pollard_rho.cpp/pollard-rho.cpp: factorización rápida.

Complejidad: `GCD: O(log n)  |  Primes: O(n log log n)  |  Miller-Rabin: O(k log² n)`

Cuándo usarlo:
GCD: inverso modular, fracciones. Criba: generar todos los primos hasta n. Miller-Rabin: verificar si un número grande es primo. Pollard-Rho: factorizar números grandes ($n > 10^{12}$).

Notas de implementación:
Miller-Rabin con las bases {2,3,5,7,11,13,17,19,23,29,31,37} es determinístico para $n < 3.3 \cdot 10^{24}$. Totient $\varphi(n) = n \cdot \Pi(1 - 1/p)$ sobre los primos p que dividen n.

`FastExpo.cc`

Exponenciación rápida (Binary Exponentiation)

Calcula a^n mod m en O(log n) usando la técnica de squaring: $a^n = (a^{(n/2)})^2$ si n es par, $a \cdot a^{(n-1)}$ si n es impar. Funciona con cualquier estructura monóidal: enteros, matrices, polinomios.

Complejidad: `O(log n)`

Cuándo usarlo:
Exponenciación modular, potencias de matrices (recurrencias lineales en $O(k^3 \log n)$), cálculo de Fibonacci(n) para $n > 10^{18}$.

Notas de implementación:
Con matrices: calcular Fibonacci, recurrencias lineales, caminos de longitud exacta en grafos. Asegurarse de que el tipo sea long long para evitar overflow en la multiplicación.

`Simplex.cc / simplex.cpp / math_simplex.cpp`

Simplex (Programación lineal)

Resuelve problemas de programación lineal: maximizar c·x sujeto a Ax <= b, x >= 0. El método simplex pivotea entre vértices del poliedro factible hasta encontrar el óptimo. math_simplex.cpp incluye manejo de degeneración.

Complejidad: `O(exponencial en el peor caso, polinomial en la práctica)`

Cuándo usarlo:
Optimización lineal con restricciones lineales, dual de LP, flujo con costos lineales, problemas de asignación con restricciones lineales.

Notas de implementación:
Para LPs pequeñas (< 1000 variables/restricciones) funciona bien en competencias. Para problemas de flujo/matching, prefer algoritmos específicos.

`modular.cpp`

Aritmética modular completa

Clase o funciones para suma, resta, multiplicación, potencia e inverso modular. Incluye cálculo de combinatorios (nCr) con factoriales precomputados y sus inversos modulares para respuestas mod p.

Complejidad: `O(log n) para inverso; O(1) resto`

Cuándo usarlo:
Prácticamente cualquier problema de conteo donde la respuesta se pide módulo un primo. Combinatoria, DP con módulo, grafos con conteo de caminos.

Notas de implementación:
Precomputar fact[] e inv_fact[] hasta el máximo n necesario. El inverso modular de a mod p (primo) es a^(p-2) mod p por el pequeño teorema de Fermat.

`math_finite_fields.cpp`

Campos finitos (GF(p^k))

Implementa aritmética en campos finitos de extensión GF(p^k) usando polinomios módulo un polinomio irreducible sobre GF(p). Soporta suma, multiplicación e inversión.

Complejidad: `O(k²) por operación`

Cuándo usarlo:
Problemas avanzados de álgebra, codificación (Reed-Solomon), criptografía en competencias, NTT

en campos no estándar.

Notas de implementación:
Raro en competencias estándar. Más común en competencias de criptografía o problemas de álgebra abstracta.

`partition.cpp`

Particiones de enteros

Calcula el número de formas de escribir n como suma de enteros positivos (función de partición p(n)). Usa la identidad del pentágono de Euler: p(n) = Σ(-1)^(k+1) [p(n-k(3k-1)/2) + p(n-k(3k+1)/2)].

Complejidad: `O(n√n) con la fórmula de Euler`

Cuándo usarlo:
Problemas de combinatoria que preguntan de cuántas formas se puede descomponer un número, problemas de monedas/cambio generalizados.

Notas de implementación:
La función de partición crece rápidamente. Para n = 100, p(100) = 190569272. Generalmente se pide módulo un primo.

`Euclid.cc`

GCD extendido / Diofánticas

El GCD extendido (Algoritmo de Euclides extendido) encuentra x, y tales que ax + by = gcd(a,b). Permite resolver ecuaciones diofánticas lineales ax ≡ c (mod b) y calcular inversos modulares.

Complejidad: `O(log min(a,b))`

Cuándo usarlo:
Inverso modular sin exponenciación, soluciones de congruencias, fracciones en módulo, teorema chino del resto (CRT).

Notas de implementación:
Para CRT: dados x ≡ a (mod m) y x ≡ b (mod n), la solución existe ssi gcd(m,n) | (b-a). Si existe, la solución es única mod lcm(m,n).

DP / Optim.

`chtrick.cpp / ds_dynamic_convex_hull.cpp`

Convex Hull Trick (CHT)

Optimiza DPs con transiciones de la forma dp[i] = min_{j<i}(dp[j] + b[j]·a[i]) donde la función objetivo son líneas y = b[j]·x + dp[j] evaluadas en x = a[i]. Mantiene el lower convex hull de las líneas y consulta por tangente.

Complejidad: `O(n) estático monótono | O(n log n) dinámico`

Cuándo usarlo:
DP de partición con costo convexo: problema del cartero chino, DP de división óptima, problemas de k-medias 1D.

Notas de implementación:
Versión estática (chtrick.cpp): si las pendientes b[j] son monótonas y los queries a[i] son monótonos, O(n) con un puntero. Si los queries no son monótonos, usar búsqueda binaria O(n log n). Para el caso dinámico usar ds_dynamic_convex_hull.cpp.

`LongestIncreasingSubsequence.cc`

LIS — Subsecuencia Creciente Más Larga

Calcula la longitud de la subsecuencia estrictamente creciente más larga de una secuencia. La versión O(n log n) mantiene un arreglo 'tails' donde tails[i] es el mínimo elemento con el que puede terminar una subsecuencia creciente de longitud i+1, usando búsqueda binaria.

Complejidad: `O(n log n)`

Cuándo usarlo:
LIS directo, reducción de LCS a LIS cuando los elementos son distintos, problemas de paciencia sorting, caja dilworth.

Notas de implementación:
Para la subsecuencia no decreciente, usar lower_bound en lugar de upper_bound. La longitud del LIS más las cadenas del dilworth = n.

`Dates.cc / Dates.java / dates.cpp`

Aritmética de fechas

Conversión entre fechas del calendario gregoriano y días julianos (número de días desde un époch). Permite calcular diferencia entre fechas, día de la semana, y aritmética de días.

Complejidad: `O(1) por operación`

Cuándo usarlo:
Problemas con fechas, diferencia de días entre fechas, día de la semana para una fecha dada.

Notas de implementación:
El número Juliano permite convertir fechas a enteros y hacer aritmética simple. Cuidado con años bisiestos: divisible por 4, excepto por 100, excepto por 400.

`yarin.cpp`

Yarin (Graph Matching / Misc)

Implementación miscelánea de matching y flujo de la colección de Yarin Gabay. Contiene variantes optimizadas de algoritmos comunes con constantes pequeñas.

Complejidad: `Variable`

Cuándo usarlo:
Como referencia alternativa de implementaciones de matching y flujo cuando las versiones principales tienen problemas.

Notas de implementación:
Verificar la fuente antes de usar en competencia para asegurarse de entender las invariantes.

`stablemp.cpp`

Stable Matching (Gale-Shapley)

Resuelve el problema del matrimonio estable: dado n hombres y n mujeres con listas de preferencias, encuentra un emparejamiento donde ningún par prefiere al otro sobre su pareja asignada (emparejamiento estable).

Complejidad: $O(n^2)$

Cuándo usarlo:
Asignación de estudiantes a universidades (NRMP), emparejamiento de residentes, cualquier problema de asignación con preferencias ordenadas.

Notas de implementación:
El algoritmo de Gale-Shapley es óptimo para el proponente (hombres en la versión clásica). Siempre termina y produce un emparejamiento estable.

Misceláneos

`IO.cc`

I/O rápido

Funciones de lectura rápida de enteros y strings usando read() en lugar de scanf/cin. Crítico en problemas con entrada grande (> 10^7 caracteres) donde el I/O estándar es el cuello de botella.

Complejidad: `O(n) lectura`

Cuándo usarlo:
Problemas con entrada masiva donde el tiempo límite es ajustado, datos de hasta 100MB.

Notas de implementación:
ios::sync_with_stdio(false) + cin.tie(nullptr) generalmente es suficiente. Solo usar I/O manual si aún hay TLE después de esto.

`RandomSTL.cc`

Aleatoriedad en STL

Uso correcto de mt19937 (Mersenne Twister) con distribuciones de la STL, y shuffle aleatorizado. Evita el uso de rand() que es lento y con distribución no uniforme.

Complejidad: `O(n) shuffle`

Cuándo usarlo:
Algoritmos aleatorizados, anti-hacking en Codeforces (randomizar el orden de procesamiento), shuffle antes de algoritmos como Welzl.

Notas de implementación:
Usar mt19937 rng(chrono::steady_clock::now().time_since_epoch().count()) para semilla basada en tiempo. uniform_int_distribution<int> para rangos específicos.

`bsearch.c`

Búsqueda binaria

Plantilla de búsqueda binaria genérica sobre una función monótona. Encuentra el menor índice donde la condición es verdadera (lower bound) o el mayor donde es falsa (upper bound).

Complejidad: `O(log n)`

Cuándo usarlo:
Búsqueda en arreglo ordenado, 'binary search on the answer' (encontrar el mínimo k tal que f(k) sea posible), búsqueda en espacio continuo.

Notas de implementación:
El patrón más útil: lo(=0), hi(=n), mientras lo<hi: mid=(lo+hi)/2; if(check(mid)) hi=mid; else lo=mid+1; respuesta = lo.

`CSP.cc`

CSP — Constraint Satisfaction Problem

Resuelve problemas de satisfacción de restricciones mediante backtracking con propagación de restricciones (arc consistency, forward checking). Incluye heurísticas como MRV (Minimum Remaining Values) y grado.

Complejidad: `Exponencial en el peor caso`

Cuándo usarlo:
Sudoku, coloración de grafos, N-reinas, problemas de scheduling con restricciones complejas que no se pueden modelar como LP o flujo.

Notas de implementación:
La eficiencia depende enormemente de las heurísticas y del problema. Para problemas estructurados, buscar reducciones a 2-SAT, flujo, o DP.

`misc_virtual_tree.cpp`

Árbol virtual (Auxiliary Tree)

Dado un árbol de n nodos y un subconjunto S de k nodos clave, construye el 'árbol virtual' mínimo que contiene solo los nodos de S y sus LCAs. Permite resolver problemas en el subárbol de S en O(k log n) en lugar de O(n).

Complejidad: `O(k log n) construcción con k nodos clave`

Cuándo usarlo:
Problemas de árbol donde las queries involucran solo k << n nodos, optimización de DPs sobre árboles con pocos nodos relevantes.

Notas de implementación:
Los nodos del árbol virtual son los k nodos clave más sus LCAs (a lo más 2k-1 nodos total). Ordenar S por DFS order (in-time) facilita la construcción.

`ds_splay_tree.cpp`

Splay Tree

BST auto-balanceado que mueve el nodo accedido recientemente a la raíz mediante rotaciones (operación 'splay'). Base del Link-Cut Tree. Soporta split y merge, y es eficiente en accesos con

localidad.

Complejidad: `O(log n) amortizado`

Cuándo usarlo:
Base para Link-Cut Trees, secuencias dinámicas con split/merge, cuando se necesitan accesos con localidad de referencia.

Notas de implementación:
El Splay Tree tiene las mismas operaciones que Treap pero con balance amortizado en lugar de esperado. Más difícil de implementar.

`bignum.cpp`

Números grandes (Bignum)

Implementa aritmética de precisión arbitraria en C++: suma, resta, multiplicación y división de enteros grandes. El repo advierte que es significativamente más lento que Python o Java BigInteger.

Complejidad: `Suma: O(n) | Multiplicación: O(n²)`

Cuándo usarlo:
Último recurso en C++ cuando se necesitan enteros de cientos de dígitos y no se puede usar Java/Python. Para multiplicación de polinomios grandes usar FFT.

Notas de implementación:
Considerar usar Java (BigInteger) o Python (int nativo) si el lenguaje está disponible. Bignum en C++ rara vez es la solución correcta en competencias modernas.

`DecFormat.java / LogLan.java`

Utilidades Java

DecFormat.java: formateo de decimales en Java con DecimalFormat y printf. LogLan.java: parser o utilidad de procesamiento de lenguaje para problemas de parsing en Java.

Complejidad: `O(1) / O(n)`

Cuándo usarlo:
Competencias donde se usa Java y se necesita formateo de salida específico o parsing de entrada compleja.

Notas de implementación:
En C++, printf con formato %.nf resuelve el formateo. Para parsing complejo en C++ preferir stringstream.

Repositorio: github.com/YeisenK/ICPC · Material de referencia para competencias ACM-ICPC